

# Clasificación del tipo de garantía usando KNN

---

**Santiago Serrano Montalvo · A01751347** Módulo 2 · Portafolio de Implementación · Técnica de aprendizaje máquina **sin** framework

---

## 1. Introducción

---

En este proyecto se implementó el algoritmo K-Nearest Neighbors (KNN) sin usar frameworks de Machine Learning. El objetivo es clasificar el tipo de garantía de créditos automotrices a partir de la información del pago mensual.

Se programó en Python. Todos los cálculos del algoritmo se hicieron a mano y se usó Matplotlib únicamente para las gráficas de resultados.

Lo que está programado desde cero: la media, la desviación estándar, la normalización, la distancia euclidiana, la búsqueda de vecinos, la votación, la matriz de confusión, el accuracy, la precisión, el recall y el F1.

Se ejecuta con:

```
python3 main.py
```

## 2. El algoritmo KNN

---

KNN es un algoritmo de clasificación que usa los datos más cercanos a un punto nuevo para decidir a qué clase pertenece. No entrena nada: solo memoriza los datos y todo el trabajo ocurre al predecir.

El funcionamiento es el siguiente:

1. Se toma un dato que se quiere predecir.
2. Se calcula su distancia con todos los datos de entrenamiento.
3. Se ordenan las distancias.
4. Se seleccionan los k vecinos más cercanos.
5. Cada vecino vota por su clase.
6. La clase con más votos es la predicción.

Para las distancias se usó la distancia euclidiana:

$$d(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

En el código es un ciclo que acumula las diferencias al cuadrado:

```
def distancia_euclidiana(a, b):  
    suma = 0.0  
    for i in range(len(a)):  
        suma += (a[i] - b[i]) ** 2  
    return math.sqrt(suma)
```

Se decidió usar  $k = 5$ .

Si hay empate en la votación gana la clase del vecino más cercano, porque la lista de vecinos ya viene ordenada por distancia y se recorre en ese orden.

## Por qué KNN

KNN no supone ninguna forma para la frontera entre las clases. No asume que se separen con una recta ni con una curva: la frontera la dibujan los propios datos. Como no se sabía de antemano cómo se relacionan el monto del crédito y el tipo de garantía, esa flexibilidad ayuda. Además su lógica es lo bastante simple para programarla completa sin librerías.

## 3. Dataset

El dataset utilizado es `interest_ledger.csv`. Tiene 10,576 registros de créditos automotrices, donde cada renglón es el pago mensual de un vehículo en un mes. El periodo va de enero 2025 a marzo 2026 y hay 4,092 vehículos distintos.

La variable a predecir es `collateral`, que es el tipo de garantía. Las clases son DEMO, FLOTILLA, NUEVO, REFACC y SEMINUEVOS.

Se usaron cinco variables:

| Variable                | Cómo se obtiene                        |
|-------------------------|----------------------------------------|
| <code>principal</code>  | columna <code>principal_amount</code>  |
| <code>interes</code>    | columna <code>interest_amount</code>   |
| <code>tasa</code>       | interés / principal                    |
| <code>antigüedad</code> | año del pago – <code>year_model</code> |
| <code>mes</code>        | mes de <code>month_date</code>         |

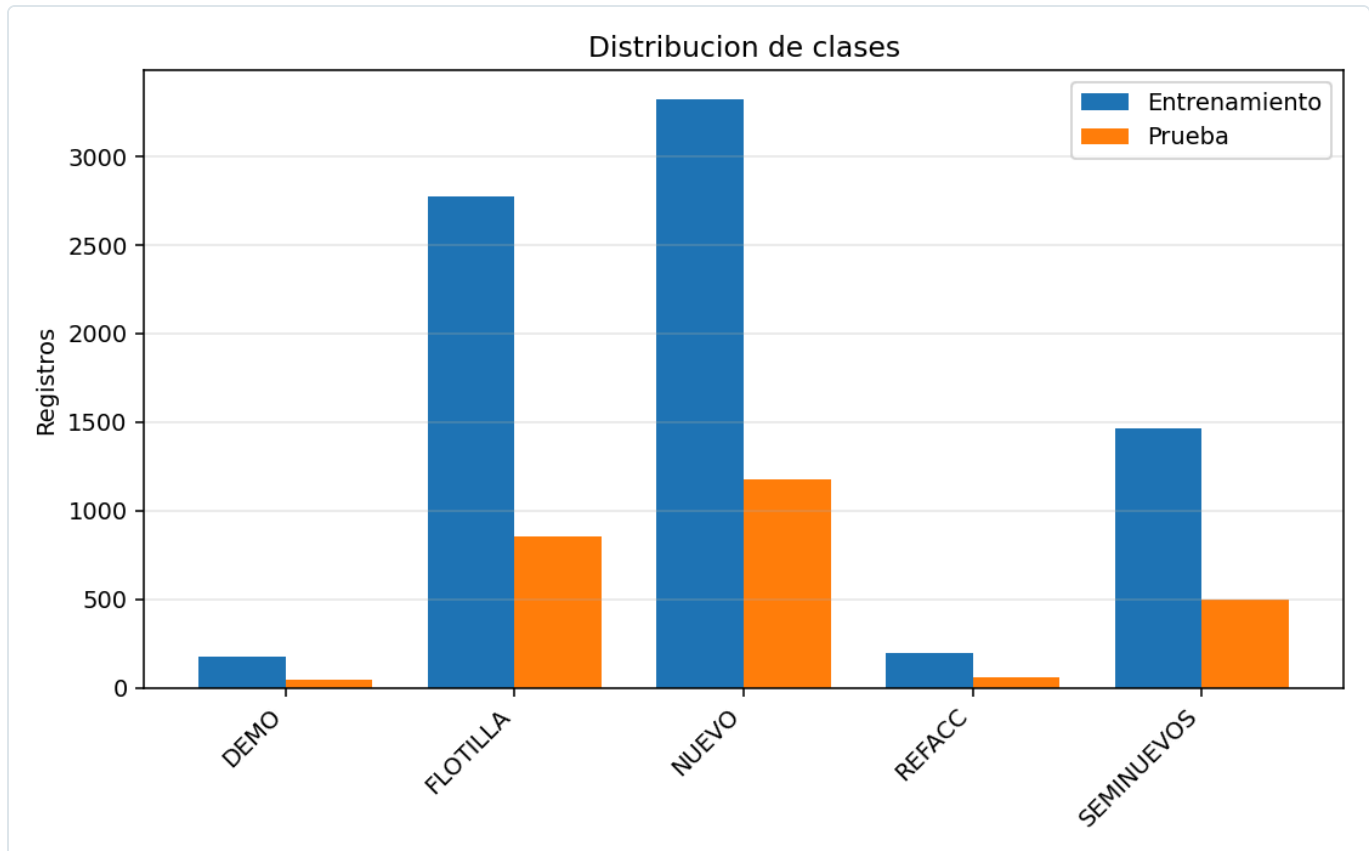
La columna `collateral` del archivo original trae espacios no separables (`\xa0`) pegados a cada valor, así que `"NUEVO\xa0"` y `"NUEVO"` se leerían como clases distintas. La carga los limpia antes de usar la etiqueta.

### 3.1 Distribución de clases

El dataset está muy desbalanceado y eso condiciona todo lo demás:

| Clase      | Registros | %      | Entrenamiento | Prueba |
|------------|-----------|--------|---------------|--------|
| NUEVO      | 4,498     | 42.5 % | 3,319         | 1,179  |
| FLOTILLA   | 3,625     | 34.3 % | 2,771         | 854    |
| SEMINUEVOS | 1,965     | 18.6 % | 1,463         | 502    |
| REFACC     | 259       | 2.4 %  | 200           | 59     |
| DEMO       | 229       | 2.2 %  | 179           | 50     |

NUEVO tiene casi 20 veces más casos que DEMO.



Distribución de clases

### 3.2 Partición

Los datos se dividieron en 75 % para entrenamiento (7,932 registros) y 25 % para prueba (2,644). Se usó `random.seed(42)` para que la división sea la misma en cada ejecución. Se barajan los índices y no las listas, para que las variables y las etiquetas sigan emparejadas.

## 4. Normalización

Antes de usar KNN se normalizaron las variables, porque algunas tienen valores mucho más grandes que otras:

| Variable   | Media      | Desviación |
|------------|------------|------------|
| principal  | 372,502.30 | 157,576.43 |
| interes    | 1,898.36   | 1,887.82   |
| tasa       | 0.0049     | 0.0037     |
| antigüedad | 0.7959     | 1.4276     |
| mes        | 5.6130     | 3.5193     |

Sin normalizar, la diferencia típica en `principal` es de cientos de miles y la de `tasa` es de milésimas, así que la distancia sería prácticamente solo el monto del crédito y las otras cuatro variables no influirían.

Se usó la estandarización z-score:

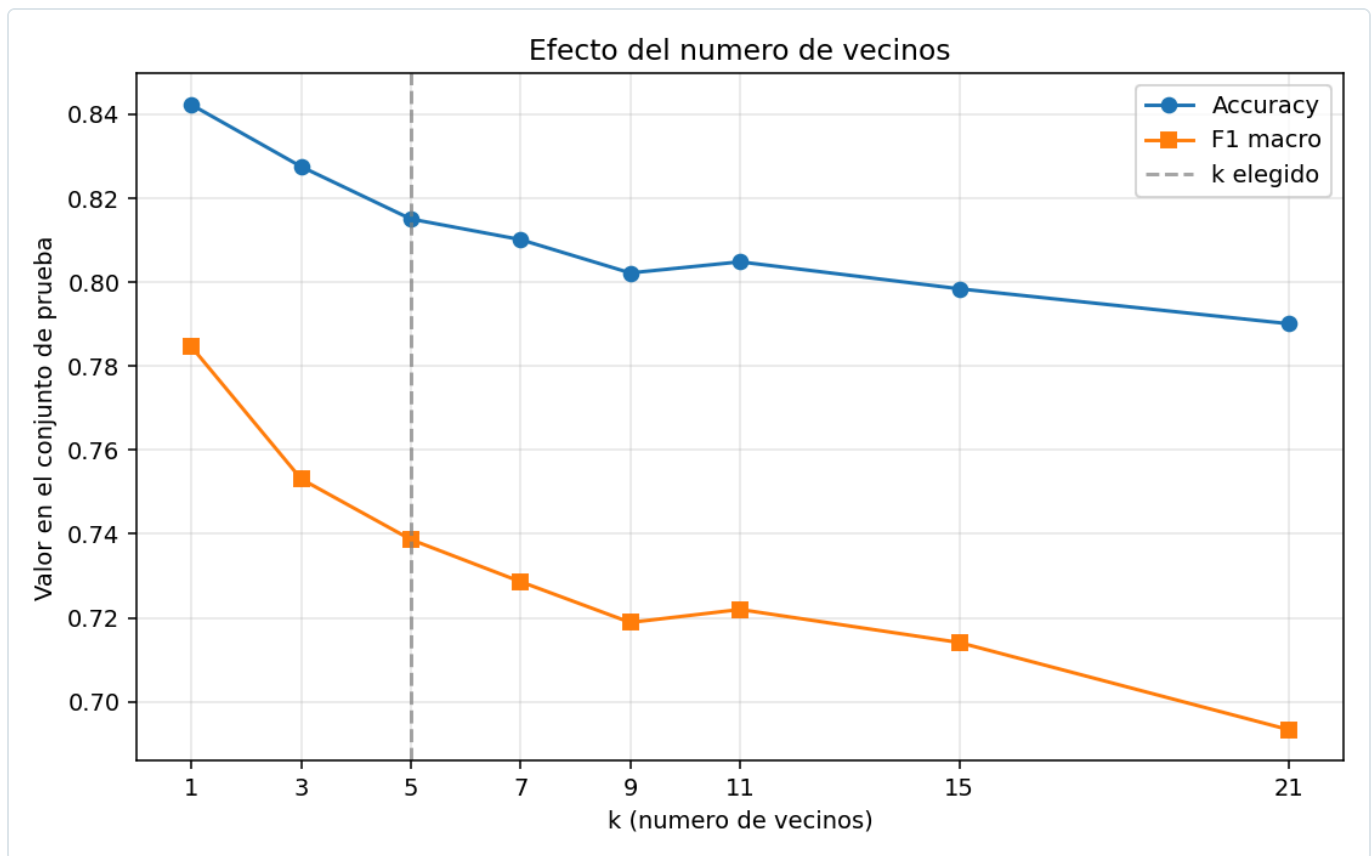
$$z = \frac{x - \text{media}}{\text{desviación}}$$

La media y la desviación se calculan **solo con los datos de entrenamiento** y esos mismos valores se aplican a prueba. Si se calcularan con todo el dataset habría fuga de información del conjunto de prueba y el resultado dejaría de ser una estimación válida.

## 5. Elección de k

Se probaron ocho valores de k sobre el conjunto de prueba. Para no repetir el cálculo completo ocho veces, el programa guarda de una sola vez los 21 vecinos más cercanos de cada punto y después solo repite la votación.

| k        | Accuracy       | F1 macro     |
|----------|----------------|--------------|
| 1        | 84.23 %        | 0.785        |
| 3        | 82.75 %        | 0.753        |
| <b>5</b> | <b>81.51 %</b> | <b>0.739</b> |
| 7        | 81.01 %        | 0.729        |
| 9        | 80.22 %        | 0.719        |
| 11       | 80.48 %        | 0.722        |
| 15       | 79.84 %        | 0.714        |
| 21       | 79.01 %        | 0.693        |



*Efecto del número de vecinos*

El accuracy baja conforme crece  $k$ . Esto no quiere decir que  $k=1$  sea el mejor modelo: el dataset tiene muchos registros casi iguales, porque el mismo vehículo aparece en varios meses con montos parecidos. Con  $k=1$  el vecino más cercano de un registro de prueba suele ser otro pago del mismo coche, así que el modelo está reconociendo vehículos en lugar de aprender la regla.

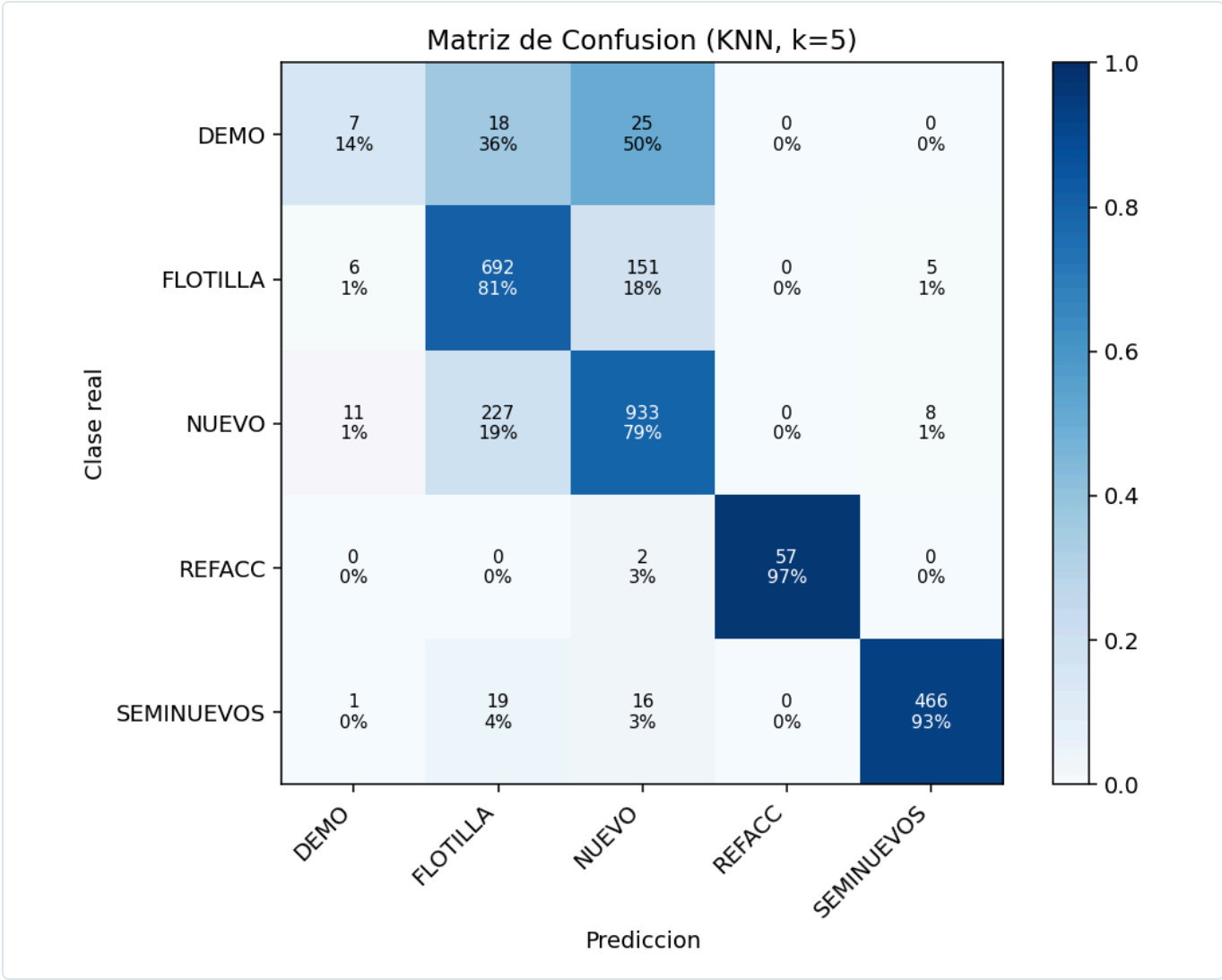
Se eligió  $k = 5$  para que la decisión se apoye en cinco puntos distintos y no en una sola coincidencia, aunque cueste 2.7 puntos de accuracy.

## 6. Resultados

Con  $k = 5$  se obtuvo un accuracy de **81.51 %** y un F1 macro de **0.739**. Un clasificador que siempre respondiera NUEVO obtendría 42.55 %, así que el modelo casi duplica esa referencia.

6.1 Matriz de confusión

| Real / Predicción | DEMO | FLOTILLA | NUEVO | REFACC | SEMINUEVOS |
|-------------------|------|----------|-------|--------|------------|
| DEMO              | 7    | 18       | 25    | 0      | 0          |
| FLOTILLA          | 6    | 692      | 151   | 0      | 5          |
| NUEVO             | 11   | 227      | 933   | 0      | 8          |
| REFACC            | 0    | 0        | 2     | 57     | 0          |
| SEMINUEVOS        | 1    | 19       | 16    | 0      | 466        |



Matriz de confusión

La mayor parte de los errores ocurre entre NUEVO y FLOTILLA.

6.2 Métricas por clase

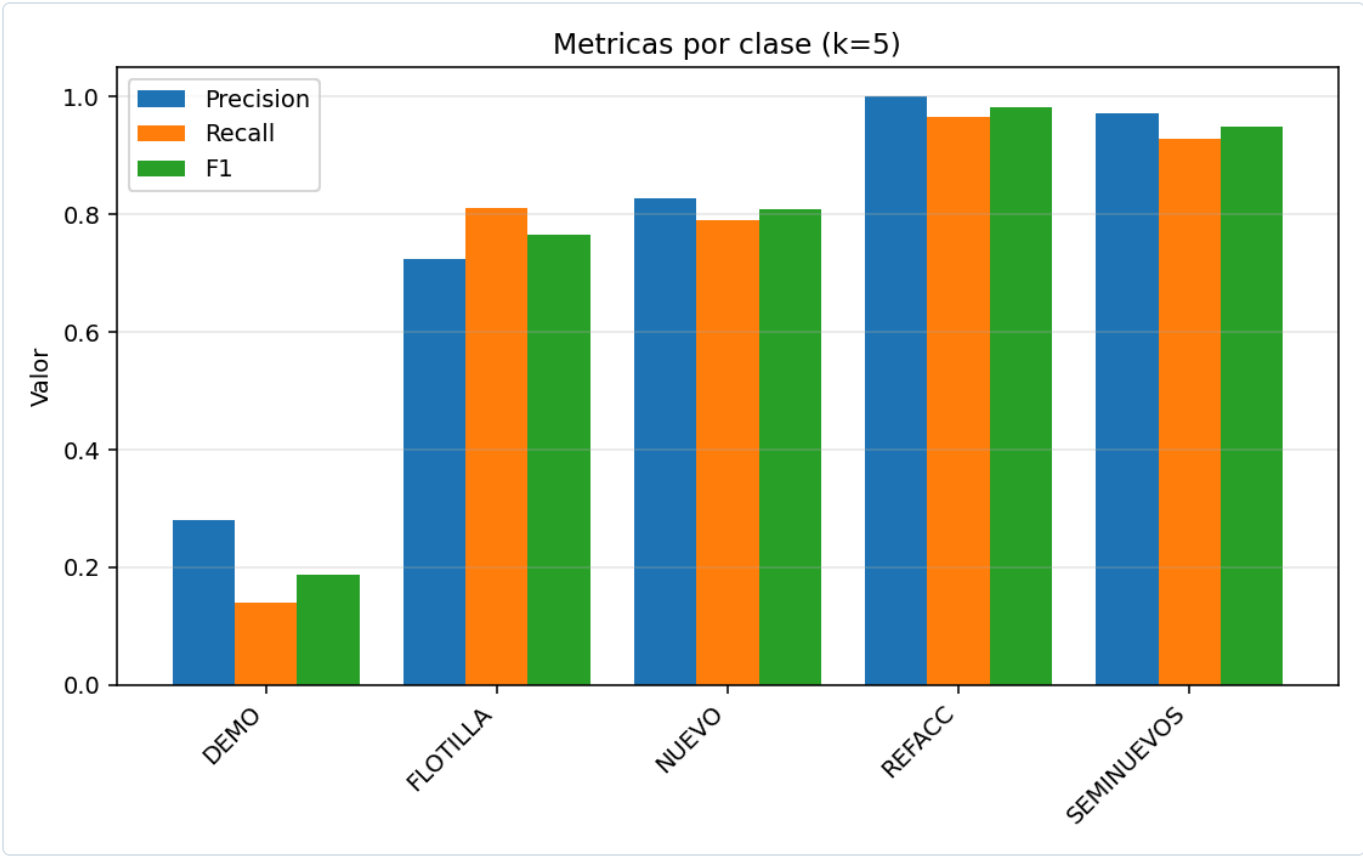
Como las clases están desbalanceadas, el accuracy solo no basta. Se calcularon precisión, recall y F1 por clase:

$$\text{precisi3n} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot p \cdot r}{p + r}$$

| Clase      | Precisi3n | Recall | F1    | n     |
|------------|-----------|--------|-------|-------|
| DEMO       | 0.280     | 0.140  | 0.187 | 50    |
| FLOTILLA   | 0.724     | 0.810  | 0.765 | 854   |
| NUEVO      | 0.828     | 0.791  | 0.809 | 1,179 |
| REFACC     | 1.000     | 0.966  | 0.983 | 59    |
| SEMINUEVOS | 0.973     | 0.928  | 0.950 | 502   |
| Macro      | 0.761     | 0.727  | 0.739 |       |



Métricas por clase

DEMO tiene apenas 50 casos en prueba, el 1.9 % del total. Un modelo que nunca predijera DEMO perdería menos de 2 puntos de accuracy pero tendría F1 de 0 en esa clase. Por eso la métrica de referencia es el F1 macro, que promedia las cinco clases con el mismo peso.

## 7. Predicciones

---

El programa termina clasificando tres créditos que no están en el CSV. Cada caso se normaliza con la misma media y desviación del entrenamiento antes de medir distancias.

```
Credito grande, auto del año
principal $350,000.00 | interes $1,500.00 | antigüedad 1 | mes 2
PREDICCION: NUEVO
votos: DEMO 2/5, NUEVO 3/5

Credito muy grande, auto nuevo
principal $1,200,000.00 | interes $9,000.00 | antigüedad 0 | mes 5
PREDICCION: DEMO
votos: DEMO 3/5, NUEVO 2/5

Credito chico, auto con 4 años
principal $90,000.00 | interes $400.00 | antigüedad 4 | mes 8
PREDICCION: SEMINUEVOS
votos: SEMINUEVOS 5/5
```

El reparto de votos dice algo por sí mismo: el tercer caso tiene 5 de 5 votos, mientras que los dos primeros se deciden 3 contra 2, es decir por un solo vecino. Se podría usar ese reparto como medida de confianza y mandar a revisión manual los casos que no lleguen a cierto umbral.

## 8. Análisis

---

**REFACC y SEMINUEVOS son las clases con mejores resultados** (F1 de 0.983 y 0.950). REFACC son créditos de refaccionaria con montos mucho más bajos que el resto, y su precisión es de 1.000: cuando el modelo dice REFACC, siempre acierta. SEMINUEVOS se distingue por la antigüedad del vehículo, que casi no se traslapa con las demás clases.

Llama la atención que REFACC tenga el mejor F1 de todas siendo una de las dos clases más chicas, con 259 registros. Eso indica que el problema de DEMO no es solo el tamaño de la muestra sino el traslape con otras clases.

**El error más grande está entre NUEVO y FLOTILLA.** Se predicen 227 NUEVO como FLOTILLA y 151 FLOTILLA como NUEVO. Juntos son 378 errores, el 77 % de todos los errores del modelo. Un crédito de flotilla es financieramente un crédito de auto nuevo: mismo tipo de vehículo, montos parecidos, antigüedad cero. Lo que los diferencia es que el acreditado es una empresa y no una persona, y esa información no está en las cinco variables. Con los datos disponibles, parte de esta confusión no se puede resolver.

**DEMO es la peor clase** (F1 de 0.187, recall de 0.140). De 50 casos el modelo solo identifica 7. Aquí se juntan dos problemas: con 179 ejemplos en entrenamiento contra 3,319 de NUEVO, es poco probable que los 5 vecinos más cercanos de un DEMO sean también DEMO, porque la votación por mayoría favorece a las clases numerosas; y además un auto de demostración es un auto nuevo que el concesionario usó unos meses, así que sus cifras son casi iguales a las de NUEVO. Los 43 errores de DEMO se reparten entre FLOTILLA y NUEVO, que son justo las clases con las que comparte perfil.

## 9. Conclusión

---

Se logró implementar KNN desde cero sin usar una biblioteca de Machine Learning. El programa lee el dataset, deriva las variables, divide los datos, los normaliza, calcula distancias, selecciona los vecinos más cercanos, vota y evalúa con matriz de confusión y métricas por clase.

Con  $k = 5$  se obtuvo 81.51 % de accuracy y 0.739 de F1 macro, contra 42.55 % de la referencia. El modelo también puede clasificar créditos nuevos desde la consola.

El desempeño no es parejo entre clases y la razón se entiende: REFACC y SEMINUEVOS ocupan regiones separadas del espacio de variables, mientras que NUEVO y FLOTILLA se confunden porque son financieramente equivalentes y lo que las distingue no está en los datos.

La normalización fue indispensable. Sin ella, `principal` con una desviación de 157,576 habría dominado por completo sobre `tasa`, con desviación de 0.0037.

Como mejoras se podría agregar el tipo de acreditado (persona o empresa), que resolvería la mayor parte de la confusión entre NUEVO y FLOTILLA; ponderar los votos por el inverso de la distancia para que los vecinos más cercanos pesen más; o ponderarlos por el inverso de la frecuencia de la clase para compensar el desbalance y subir el recall de DEMO.

---

Para reproducir los resultados: `cd M2_SinFramework && python3 main.py`. Requiere Python 3 y matplotlib. Las figuras quedan en `figuras/` y la salida completa en `salida.txt`.